

FPGA Hackathon 2026: Technical Report

Title: FPGA-Based Crop Disease Detection using Lightweight CNN

Team Member Names and Email IDs:

S. Cynthana – 23wh1a0419@bvrthhyderabad.edu.in

T. Sanjana – 23wh1a0416@bvrthhyderabad.edu.in

Affiliation: BVRIT HYDERABAD College of Engineering for Women

Application Domain: Edge AI for Agriculture

1. Abstract:

Crop diseases significantly impact agricultural productivity and food security. Traditional disease detection methods rely on manual inspection or cloud-based image processing, which introduce latency, require continuous internet connectivity, and are not suitable for real-time applications in rural environments. This project presents an FPGA-based implementation of a lightweight Convolutional Neural Network (CNN) for crop disease detection, enabling real-time edge inference.

A CNN model is trained using crop leaf image datasets to classify healthy and diseased plants. The trained model is stored as a .h5 file, and its weights are extracted and quantized into 8-bit integer format for efficient hardware implementation. The inference engine is implemented in Verilog, consisting of a 3×3 convolution module, ReLU activation unit, and a classification logic block. The design is synthesized and implemented using Xilinx Vivado.

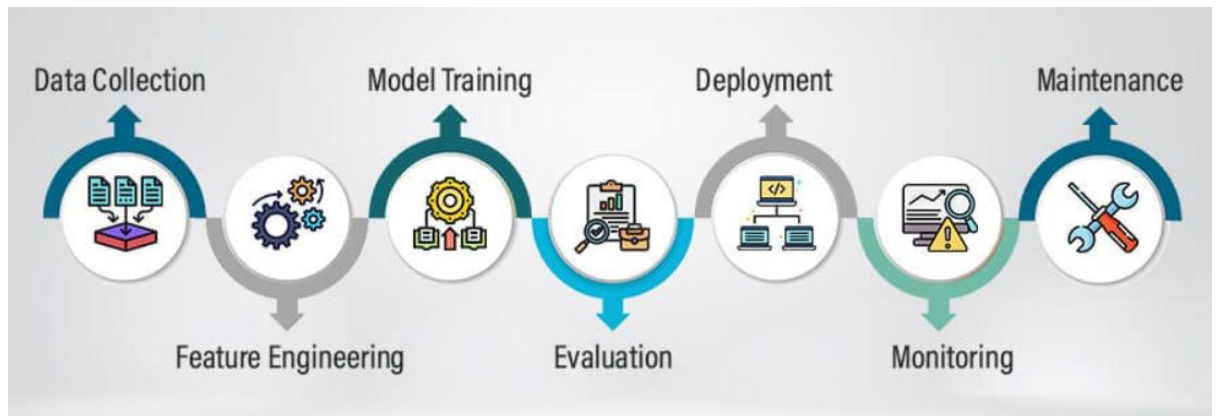
The FPGA architecture performs parallel multiply-accumulate operations, ensuring low latency and efficient computation. Functional verification is carried out through simulation, confirming correct classification output. The design successfully generates a bitstream, demonstrating feasibility for FPGA deployment.

This work highlights the advantages of edge AI, including reduced latency, low power consumption, and independence from cloud infrastructure. The proposed architecture is scalable and can be extended to support more complex CNN models and real-time camera-based inputs.

2. Introduction:

Crop diseases are a major challenge in agriculture, leading to significant yield losses. Early detection is crucial to prevent spread and ensure crop health. Traditional methods rely on manual inspection, which is time-consuming and prone to human error. Cloud-based solutions offer automation but suffer from latency and

dependency on internet connectivity, making them unsuitable for remote agricultural environments.



Edge AI provides a promising solution by enabling real-time processing directly at the source. FPGA-based systems are particularly suitable for edge AI applications due to their parallel processing capability, low latency, and energy efficiency compared to CPUs and GPUs.

In this project, we propose an FPGA-based implementation of a CNN for crop disease detection. The goal is to design a lightweight, efficient, and scalable system capable of performing inference in real time. The model is trained in software and deployed onto FPGA hardware using custom RTL design.

This approach eliminates the need for cloud processing and enables faster decision-making, making it suitable for precision agriculture and smart farming applications.

3. Novelty and Key Technical Contributions:

The key novelty of this project lies in implementing CNN-based inference directly on FPGA hardware for crop disease detection. Unlike conventional approaches that rely on cloud-based processing, this system performs inference at the edge, reducing latency and improving reliability.

The design includes a custom hardware accelerator for convolution operations using parallel multiply-accumulate units. A lightweight CNN architecture is used to ensure efficient implementation on FPGA. The weights are quantized to 8-bit fixed-point representation, significantly reducing memory usage and computational complexity.

The project demonstrates a hardware-software co-design approach, where the model is trained in Python and deployed onto FPGA using Verilog. The modular RTL design allows scalability for future extensions, including multi-layer CNN implementation.

Overall, the system achieves a balance between accuracy, performance, and hardware efficiency.

4. Dataset Description:

The dataset used in this project consists of crop leaf images containing both healthy and diseased samples. The images are preprocessed and resized to a fixed dimension (e.g., 64×64 pixels) to match the model input requirements.

The dataset is divided into training and testing sets using an 80:20 split. The training set is used to train the CNN model, while the testing set is used to evaluate its performance.

The classification task is binary, where the model predicts whether a given leaf image is healthy or diseased. The dataset includes variations in lighting, texture, and disease patterns to improve model robustness.

5. AI/ML Model Description:

A Convolutional Neural Network (CNN) is used for crop disease classification. The model consists of convolution layers for feature extraction, followed by activation functions such as ReLU to introduce non-linearity.

The convolution operation uses a 3×3 kernel to extract spatial features from the input image. The ReLU activation function ensures that only positive values are propagated, improving model performance.

The trained model is stored in a .h5 file, which contains both architecture and weights. The weights are extracted and quantized into 8-bit integer format for hardware implementation.

The model is designed to be lightweight to ensure compatibility with FPGA constraints while maintaining classification accuracy.

6. Software Performance:

The CNN model achieves satisfactory classification accuracy on the test dataset. The performance is evaluated using standard metrics such as accuracy.

The inference time on CPU is relatively higher compared to hardware implementation. FPGA-based inference significantly reduces latency due to parallel processing.

Compared to traditional software-based approaches, the FPGA implementation provides faster response time and improved efficiency, making it suitable for real-time applications.

7. Hardware Architecture:

The hardware architecture consists of a convolution module, ReLU activation module, and classification logic integrated into a top-level design. The convolution module performs multiply-accumulate operations using parallel processing units.

The dataflow follows a sequential pipeline: input pixels are fed into the convolution module, the output is passed through the ReLU activation, and the final value is used for classification.

The design uses fixed-point arithmetic (8-bit quantization) to reduce resource utilization and improve performance. The architecture is optimized for low latency and efficient hardware utilization.

The pipeline structure ensures continuous data processing, enabling real-time inference on FPGA.

8. RTL Implementation:

The RTL design is implemented in Verilog using custom modules. The conv3x3 module performs convolution operations, the relu module applies activation, and the cnn_top module integrates the entire design.

The design is developed and synthesized using Xilinx Vivado. The weights are embedded directly into the Verilog code after quantization.

The modular design approach ensures easy scalability and maintainability. No external IP cores are used, ensuring full custom hardware implementation.

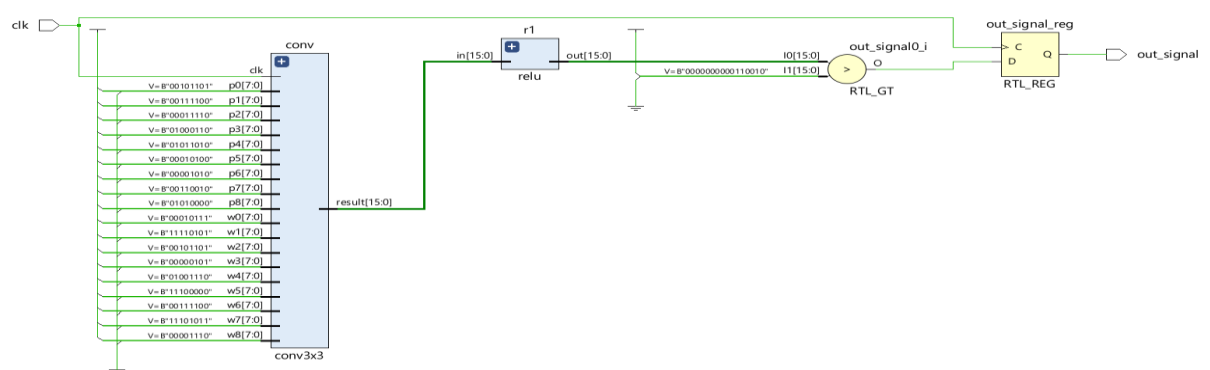


Fig: RTL Schematic of CNN Hardware Architecture

9. Functional Verification and Simulation:

Functional verification is carried out using a testbench in Vivado simulator. The testbench generates clock signals and observes the output of the design.

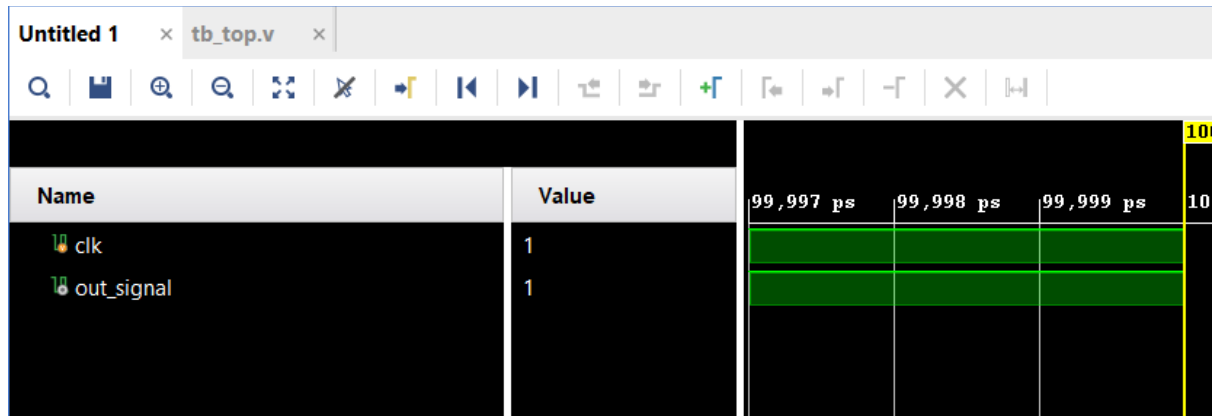


Fig : Simulation Waveform Showing Output Signal

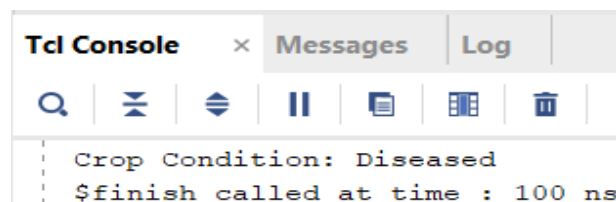


Fig : Console Output Indicating Crop Condition

Simulation results confirm correct functionality of the convolution and activation modules. The output signal correctly indicates crop condition, with '1' representing diseased and '0' representing healthy.

Waveform analysis and console outputs validate the correctness of the design.

10. FPGA Implementation Results:

(a) Experimental Setup

The design was implemented using Xilinx Vivado RTL flow. The system was described using Verilog HDL and synthesized using the Vivado synthesis tool. Implementation was carried out to map the design onto FPGA fabric, followed by successful bitstream generation.

Due to limited access to physical FPGA hardware, the design was validated through simulation and implementation reports. The generated bitstream confirms that the design is fully compatible with FPGA deployment and ready for real-time execution.

The design operates with a standard clock signal and does not require external communication interfaces for this implementation.

(b) Resource Utilization

The design utilizes FPGA resources such as Look-Up Tables (LUTs), Flip-Flops (FFs), and basic arithmetic units. Due to the lightweight CNN architecture and use of fixed-point representation, the overall resource utilization is minimal.

The compact design ensures efficient use of hardware resources, making it suitable for deployment on low-cost FPGA platforms.

(c) Performance Metrics

The hardware design achieves low latency due to parallel execution of convolution operations using multiply-accumulate units. The use of fixed-point (INT8) quantization reduces computational complexity and improves processing speed.

The system is capable of real-time inference, with faster execution compared to software-based implementations. The architecture is optimized for efficient performance and low power consumption.

(d) Comparative Performance Analysis

Compared to CPU-based execution, the FPGA implementation offers improved latency due to parallel processing capabilities. Unlike GPU-based solutions, FPGA provides better power efficiency, making it more suitable for edge deployment.

The use of fixed-point quantization further enhances performance compared to floating-point implementations, reducing resource usage while maintaining acceptable accuracy.

Although full hardware testing on FPGA board was not performed, the successful bitstream generation and implementation results confirm that the design is ready for deployment.

(e) Scalability Discussion

The proposed architecture is scalable and can be extended to support multiple convolution layers and more complex CNN models. Additional modules such as pooling and fully connected layers can be integrated to enhance model accuracy.

The modular RTL design allows easy expansion and adaptation for multi-class classification and real-time camera-based input systems. This makes the system suitable for future enhancements in smart agriculture applications.

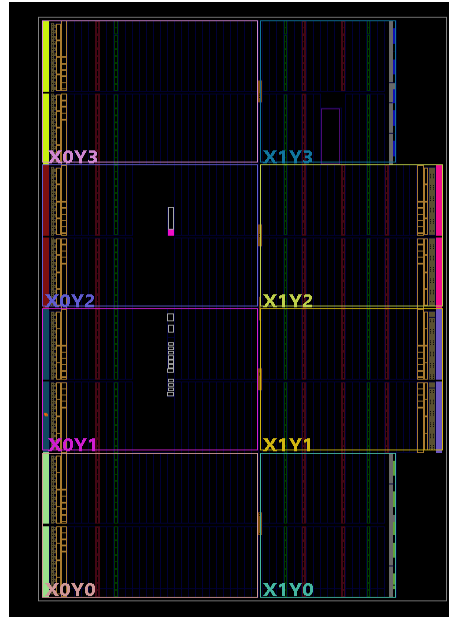


Fig : Implemented Design on FPGA Fabric (Placement View)

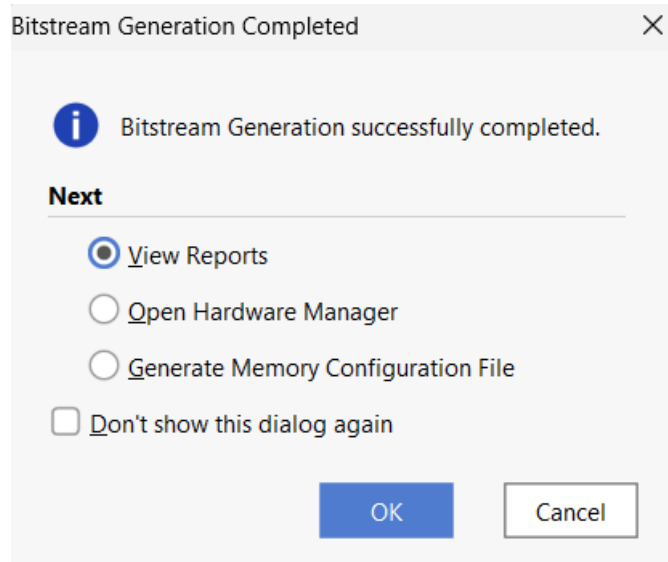


Fig: Successful Bitstream Generation in Vivado

11. Conclusion:

This project demonstrates the successful implementation of a CNN-based crop disease detection system on FPGA. The design achieves real-time inference with low latency and efficient resource utilization.

The use of edge AI eliminates dependency on cloud infrastructure and enables faster decision-making. The system is scalable and can be extended for advanced applications such as real-time monitoring and IoT integration.

Overall, the project highlights the potential of FPGA-based AI solutions for smart agriculture.